

Real-Time Systems & Streaming

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles

Every request/response pattern this program has covered assumes the client asks and the server answers, once, per request. But a live chat message, a driver's moving location dot, a live cricket score, or a typing indicator all need the *server* to push new data to the client the moment it's available, without the client having to ask again and again. This chapter covers the three standard techniques for this, in increasing order of real-time capability and implementation complexity.

Short polling and long polling

Short polling is the naive baseline: the client sends a normal HTTP request every few seconds asking "anything new?" and the server responds immediately, with or without new data. It's trivially simple to implement but wasteful — most polls return "nothing new," burning server resources and battery/bandwidth on the client for no benefit, and it introduces up to a full polling-interval's worth of latency before new data is even seen. **Long polling** improves on this: the client sends a request, and instead of responding immediately, the server *holds the connection open* until either new data becomes available or a timeout is reached, then responds and the client immediately opens a new long-poll request. This meaningfully reduces both wasted requests and latency compared to short polling, while still working over plain HTTP with no special infrastructure — but it still requires opening a fresh connection after every response, and holding many concurrent long-poll connections open still consumes real server resources (a connection/thread per waiting client, in naive implementations).

Server-Sent Events (SSE)

Server-Sent Events is a standard built specifically for one-directional, server-to-client streaming over a single, long-lived HTTP connection — the server can push any number of events over time without the client re-requesting, and the browser's built-in **EventSource** API handles automatic reconnection if the connection drops. SSE is simpler to implement and operate than a full bidirectional protocol precisely because it only needs to solve the server-to-client direction, making it an excellent fit for feeds, live notifications, progress updates, and dashboards where the client never needs to send anything back over the same channel.

WebSockets

WebSockets provide genuine full-duplex communication: after an initial HTTP handshake (the connection starts as HTTP and "upgrades" to the WebSocket protocol), both the client and server can send messages to each other at any time, over a single persistent TCP connection, with lower per-message overhead than repeated HTTP requests. WebSockets are the right choice when both directions are genuinely needed with low latency — a chat app (both sending and receiving messages continuously), collaborative editing (cursor positions, live edits flowing both ways), multiplayer game state.

Backend engineer analogy

Short polling is like repeatedly calling a friend every 5 minutes to ask 'anything new?' Long polling is like calling once and asking them to stay on the line until they actually have news, then hanging up and calling back. SSE is like subscribing to a one-way announcement channel where they talk and you just listen. WebSockets is an open phone line where either of you can speak at any moment. Each step trades more infrastructure complexity for lower latency and richer interaction.

The real operational cost: connection state at scale

The property that makes WebSockets (and to a lesser extent, long-lived SSE and long-polling connections) genuinely harder to scale than plain stateless HTTP APIs (Phase 1 Chapter 9's statelessness principle) is that each open connection is **stateful** — it's pinned to one specific server process for its entire lifetime. This breaks the simple "any request can go to any server behind the load balancer" model: if a client's WebSocket connection is open on Server A, and you need to push a message to that client from logic running on Server B (because your app is horizontally scaled across many servers), Server B can't just handle it directly — you need a way to route "send this to user 42" to whichever specific server currently holds user 42's connection. This is typically solved with a **pub/sub backplane** (often Redis Pub/Sub, or Kafka, Phase 2 Chapter 8): every server subscribes to relevant channels, and when Server B needs to notify user 42, it publishes to a channel that Server A (holding the actual connection) is listening on, which then forwards the message down that specific WebSocket.

This connection-affinity problem also directly interacts with load balancing (Phase 1 Chapter 9) and service discovery (Phase 2 Chapter 12): load balancers fronting WebSocket traffic need **sticky sessions** so reconnects land back on a server that can resume state cleanly, and horizontally scaling a WebSocket-serving fleet means accepting that connections are unevenly distributed and that a server restart or deploy will drop and force-reconnect every client it was holding — a real operational consideration that stateless HTTP APIs simply don't have to think about.

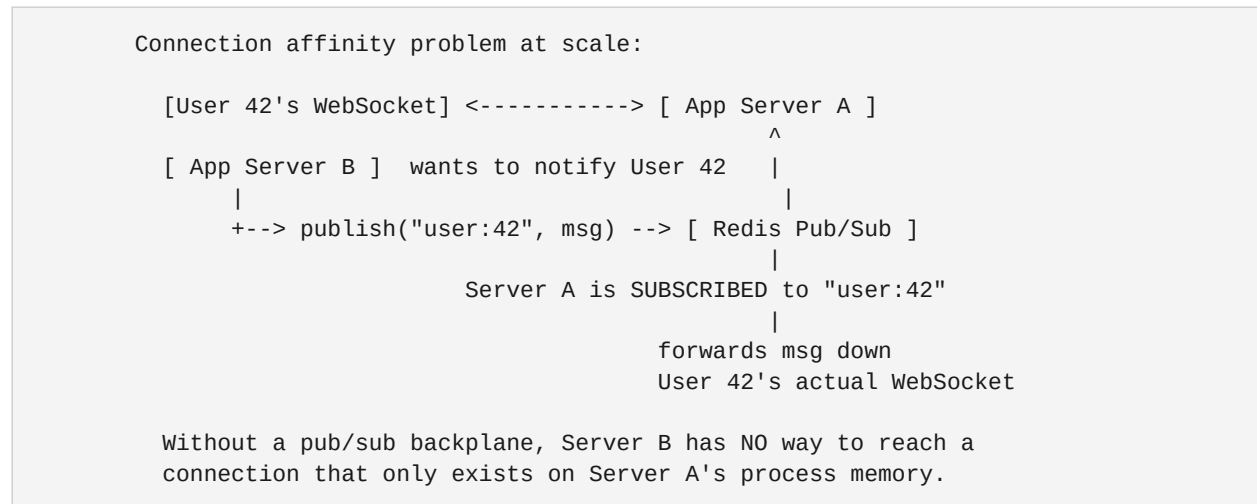
2. Real-World Example / Case Study

Careem's and Talabat's live driver-tracking maps use WebSockets (or a comparable persistent-connection technology) to push location updates to the rider's app in real time, with a Redis Pub/Sub or similar backplane routing "driver 8821 moved" events to whichever specific app server instance currently holds that rider's live connection.

Swiggy's and Zomato's order-status tracking screen ("your order is being prepared... out for delivery...") is a good SSE candidate rather than full WebSockets — the client only ever needs to receive status updates, never send anything back over that same channel, making SSE's simpler, one-directional model a better operational fit than paying for WebSockets' full-duplex complexity unnecessarily.

A Razorpay or PhonePe merchant dashboard showing live transaction notifications might reasonably start with long polling for simplicity in an early product stage, and migrate to SSE or WebSockets only once genuine scale or latency requirements (many concurrent merchants needing sub-second updates) justify the added infrastructure — a good illustration that this chapter's techniques form a real progression, not a single "correct" answer for every real-time need.

3. Diagram



4. Trade-offs Table

Technique	Direction	Latency	Operational cost
Short polling	Client asks repeatedly	Up to poll-interval delay	Low complexity, wasteful requests
Long polling	Client asks, server holds	Near-real-time	Moderate — many held-open connections
Server-Sent Events	Server -> client only	Real-time	Moderate — simpler than WebSockets
WebSockets	Full-duplex, both ways	Real-time, lowest overhead	Highest — stateful connections need a pub/sub backplane

5. Common Interview Questions

Q1: "Design the real-time layer for a live chat app. What technology do you choose, and why?"

Model approach: Recommend WebSockets, justified by the genuine bidirectional need (sending and receiving messages continuously), then address the connection-affinity/scaling problem directly with a Redis Pub/Sub or Kafka backplane and sticky-session load balancing.

Q2: "Why might you choose SSE over WebSockets for a notification feed?"

Model approach: Point out that notifications only flow server-to-client, so SSE's simpler one-directional model avoids the operational cost of full-duplex WebSockets for a use case that never needed the client-to-server direction in the first place.

Q3: "How does a horizontally-scaled app handle pushing a message to a specific user's WebSocket connection?"

Model approach: Explain the stateful-connection problem directly — the connection lives on one specific server process — and describe the pub/sub backplane solution, where any server can publish a message that the server actually holding the connection picks up and forwards.

Q4: "What happens to open WebSocket connections during a deploy?"

Model approach: Acknowledge directly that a deploy or restart of a server holding live connections drops them, forcing clients to reconnect (ideally to a different, healthy instance), and mention this as a real operational trade-off of stateful connections versus stateless HTTP APIs, sometimes mitigated with graceful connection draining during deploys.

6. Practice Exercises

- 1 For each of the following, choose short polling, long polling, SSE, or WebSockets, with justification: a stock ticker widget, a multiplayer game, a 'new email' badge counter, a live sports score.
- 2 Design the pub/sub backplane architecture needed to notify a specific user's WebSocket connection when your app is horizontally scaled across 10 servers.
- 3 Explain why sticky sessions matter for a WebSocket-serving load balancer, and what happens if they're misconfigured.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Long Polling vs SSE vs WebSockets vs QUIC Simply Explained"	Clear side-by-side comparison of all the techniques in this chapter.
English	"Real-Time Techniques: Polling, Long Polling, Server-Sent Events & WebSockets"	Reinforces the progression from simplest to most real-time.
Hindi	"Short Polling vs Long Polling vs WebSockets System Design Interview Distributed Systems in Hindi"	Hindi comparison focused on the interview-relevant trade-offs.
Hindi	"Long polling, server sent Events and Web Sockets Real time communication jargons in Hindi" — Chai aur Code	Hindi walkthrough of all three real-time techniques together.

8. Key Takeaways

- Short polling is simple but wasteful; long polling reduces waste by holding the connection open until data is ready.
- SSE gives efficient, real-time, one-directional (server-to-client) streaming with built-in reconnection.
- WebSockets give full-duplex, low-overhead, real-time communication when both directions are genuinely needed.
- WebSocket (and long-lived) connections are stateful and pinned to a specific server, breaking the stateless-request model.
- A pub/sub backplane (Redis, Kafka) is needed to route messages to a user's connection from any server in a horizontally-scaled fleet.
- Sticky sessions and graceful connection draining are real operational concerns for stateful real-time connections.

- Choose the simplest technique that meets the actual latency and directionality requirement, not the most impressive-sounding one.